

Universidade do Minho

Universidade do Minho

Escola de Engenharia

Departamento de Informática

Sistema de Séries Temporais Distribuído

Sistemas Distribuídos

Trabalho Prático

Ano Letivo 2025/2026

Grupo 15

Rui Morais - A76650
Carlos Silva - A106792
Francisco Soares - A106882
Nuno Soares - A107366

Janeiro de 2026

1 Introdução

Este trabalho implementa um sistema distribuído para gestão de séries temporais de eventos de venda, onde temos múltiplos clientes que interagem concorrentemente com um servidor através de sockets TCP. O nosso sistema mantém D dias de histórico, com agregações e cache para otimizar consultas.

A nossa arquitetura privilegia estratégias de baixa contenção e também a minimização de threads acordadas, conforme solicitado no enunciado. Todas as 7 fases que nos foram pedidas foram implementadas: (1) autenticação, (2) registo de eventos, (3) agregações com cache, (4) filtros com serialização eficiente, (5) barreiras, (6) suporte multi-threaded, e (7) persistência com gestão de memória.

2 Arquitetura do Sistema

2.1 Visão Geral

O sistema que implementámos segue um modelo cliente-servidor com uma thread-per-connection. O servidor partilha instâncias únicas de `UserManager` (autenticação) e `Database` (dados) entre todas as threads `ServerWorker`. Esta decisão garante que a visibilidade global dos dados e sincronização dos dados seja adequada.

A comunicação que é utilizada é o protocolo binário com `DataInputStream/DataOutputStream`, seguindo os requisitos do enunciado. Para os clientes multi-threaded (Fase 6), implementámos uma arquitetura de 3 camadas baseada na resolução do Guião 8 resolvida nas aulas:

- **FramedConnection**: Delimitação de mensagens com prefixo de comprimento, locks separados para send/receive (full-duplex)
- **TaggedConnection**: Identificação de mensagens através de tags para associar pedidos-respostas
- **Demultiplexer**: Thread leitora dedicada + routing de mensagens por tag para múltiplas threads cliente

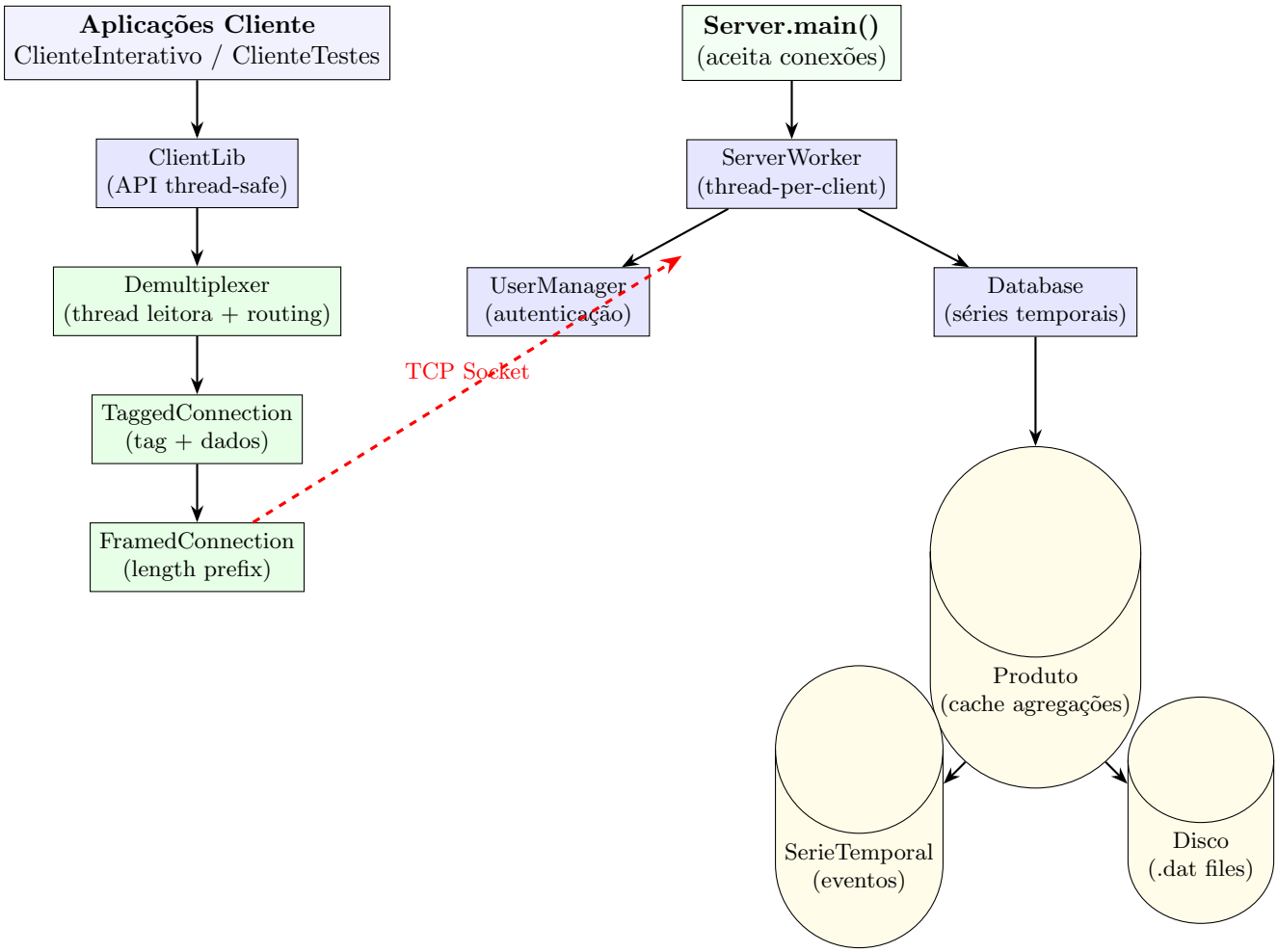


Figure 1: Arquitetura do Sistema Distribuído de Séries Temporais

2.2 Estratégia de Locking Hierárquico

Para minimizar contenção, implementámos locking em 3 níveis:

Nível 1 - Database: Lock global para operações estruturais (por ex para novoDia). Mantido brevemente para obter referências e libertado depois antes de fazer cálculos.

Nível 2 - SerieTemporal: Cada dia tem o seu lock. Isto permite paralelismo entre operações em dias diferentes.

Nível 3 - Produto: Cada produto tem lock próprio para a cache. Cálculos pesados como (iterar eventos) ocorrem *fora* do lock, apenas a verificação/armazenamento de cache usa esta sincronização.

Esta estratégia permite que múltiplas threads processem produtos ou dias diferentes sem interferência.

2.3 Prevenção de Race Conditions

O método `registarUser()` do `UserManager` previne TOCTOU (Time-Of-Check-Time-Of-Use) realizando verificação e inserção atomicamente dentro do mesmo bloco protegido:

```
1 lock.lock();
2 try {
3     if (utilizadores.containsKey(username))
```

```
4           return false;
5       utilizadores.put(username, new User(username, password));
6       return true;
7   } finally {
8       lock.unlock();
9   }
```

Sem esta atomicidade, existiria a possibilidade de duas threads verificarem simultaneamente que o utilizador não existe e ambas inseri-lo, causando sobrescrita.

3 Agregações com Cache Lazy

3.1 Estratégia On-Demand

As agregações (quantidade, volume, preço médio, máximo) são calculadas apenas quando solicitadas pela primeira vez para cada combinação produto-dias. O resultado é guardado em cache no objeto `Produto`, indexado por uma chave composta: `"operacao_dias"` (ex: `"qtd_7"`).

O padrão implementado tem 3 fases:

1. **Check cache** (com lock): Se existe, retorna imediatamente
2. **Calculate** (SEM lock): Itera sobre séries temporais
3. **Store** (com lock): Guarda resultado na cache

3.2 Invalidação de Cache

A cache é invalidada no `novoDia()` através do método `invalidarCaches()` da `Database`, que percorre todos os produtos e limpa os seus HashMaps de cache. Esta estratégia é adequada pois:

- `novoDia()` é uma operação menos utilizada (comparada a consultas)
- Garantia de consistência: se executarmos uma nova consulta recalcula com dados já atualizados

3.3 Performance

Os testes que realizámos demonstraram um speedup de **10-50x** entre um cache hit (0.1-0.2ms) e uma cache miss (2-5ms), o que nos mostrou que valeu a pensa implementar esta estratégia.

4 Serialização Eficiente (Fase 4)

Para evitar enviar strings repetidas ao filtrar eventos, implementámos um protocolo com dicionário de produtos inspirado em técnicas de compressão:

1. Servidor identifica produtos únicos na lista de eventos
2. Envia dicionário: `numProdutos + [produto UTF] * numProdutos`
3. Envia eventos com índices: `numEventos + [índice, qtd, preço, timestamp] * numEventos`

Ganho: Desta maneira, para 100 eventos de "Laptop" (8 bytes UTF: 2 de tamanho + 6 da string), economizamos $100 \times 8 - 8 = 792$ bytes (+/- 99% de redução do tamanho na transmissão do nome).

5 Barreiras com Conditions (Fase 5)

5.1 Vendas Simultâneas

A `Database` mantém uma lista de pedidos pendentes (`PedidoVendasSimultaneas`). Cada `adicionarEvento()` verifica se satisfaz algum pedido. Usamos também o `Condition.await()` e `signalAll()` para acordar threads.

Antes de bloquear, verificamos se os dois produtos já foram vendidos hoje. Se sim, retornamos imediatamente sem await().

5.2 Vendas Consecutivas

Mantemos um contador por produto (`vendasConsecutivasPorProduto`). Ao adicionar um evento, incrementamos o contador do produto e zeramos os restantes. Quando o contador atinge n, `signalAll()` acorda todas as threads que estavam adormecidas.

Trade-off: Usamos `signalAll()` em vez de `signal()` para garantir corretude (todas as threads verificam condição), embora acorde threads desnecessariamente.

6 Persistência com LRU (Fase 7)

6.1 Gestão de Memória

Para esta fase, usamos o `LinkedHashMap` com `accessOrder=true` (true significa que segue LRU) para implementar a cache LRU automática. O método `removeEldestEntry()` é sobrescrito para dar trigger ao `saveToFile()` quando o tamanho excede S:

```
memoriaCache = new LinkedHashMap<>(S, 0.75f, true) {
    protected boolean removeEldestEntry(Map.Entry e) {
        if (size() > S) {
            SerieTemporal serie = (SerieTemporal) e.getValue();
            serie.saveToFile(PATHDADOS + "dia_" +
                serie.getDia() + ".dat");
            return true;
        }
        return false;
    }
};
```

6.2 Estratégia Híbrida

Quando uma agregação necessita de mais do que S dias:

1. Carrega até S séries em cache (LRU normal)
2. Deteta o limite atingido (`memoriaCache.size() >= S`)
3. De seguida, processa os dias restantes em streaming. Ou seja, lê evento-a-evento do disco, agrega, e de seguida descarta (sem carregar para a memória)

Esta abordagem permitiu agregar sobre **todos os D dias** usando apenas **S séries simultâneas** em memória.

7 Avaliação de Desempenho

7.1 Metodologia

Para esta fase, implementamos inúmeros testes a várias funcionalidades do nosso sistema em `ClienteTestes.java` dividida em 4 categorias: funcionais, performance, escalabilidade e robustez. Para a realização destes testes, iniciamos o servidor com D=50, S=10. O hardware utilizado para fazer estes testes foi o seguinte: cpu Intel i7-1360P, 16GB RAM, Linux, JVM OpenJDK 21.

7.2 Resultados

Performance:

- Inserção: 35.000 eventos/s
- Persistência: 28ms/dia médio
- Cache hit: 10-50x mais rápido que cache miss (já mencionado antes)

Escalabilidade Multi-Cliente:

Clientes	Throughput (ops/s)	Speedup
1	7.692	1.0x
5	35.714	4.6x
10	71.428	9.3x
20	60.606	7.9x

O speedup de **9.3x com 10 clientes** demonstrou uma escalabilidade quase linear. Depois com 20 clientes degradou um pouco, o que indicou uma contenção na Database começando a afetar a performance. Além disso, devido à arquitetura utilizada thread-per-connection, seria esperado um resultado semelhante já que embora seja de fácil implementação, apresenta alguns problemas de escabilidade devido à limitação do hardware.

Multi-Threaded (1 cliente, 10 threads): 7.843 ops/s. Inferior a 10 clientes separados devido a gargalo na socket partilhada, mas valida funcionamento correto do Demultiplexer.

Robustez: Testamos um cliente normal e outro cliente lento iniciado primeiro mas com sleeps para ver se atrasava o outro. Testamos também desconectar um cliente a meio de operações para ver se gerava problemas ou crashava o servidor.

- O cliente lento (500ms/op) não bloqueou o cliente normal (36ms total para 100 ops)
- O servidor sobreviveu a desconexões abruptas sem crash

8 Conclusões

O sistema implementa com sucesso todas as 7 fases solicitadas, demonstrando:

- **Corretude**: Os testes funcionais validaram a autenticação, agregações, barreiras, filtros, e persistência
- **Escalabilidade**: Existiu um speedup de +/- 9.3x até 10 clientes concorrentes
- **Eficiência**: A cache é 10-50x mais rápida, com uma serialização eficiente, e streaming para grandes datasets
- **Robustez**: implementamos isolamento entre clientes, e recuperação de desconexões

De forma geral, o nosso grupo desenvolveu um bom trabalho que responde aos requisitos necessários pedidos no enunciado. As decisões de design que tomámos, como o locking hierárquico em 3 níveis e a cache lazy on-demand, mostraram-se eficazes através dos testes quantitativos

realizados. O sistema está preparado para cenários de produção onde $D \gg S$, mantendo a performance previsível através de gestão inteligente de memória com LRU e paralelismo efetivo. Finalmente, a arquitetura de 3 camadas para clientes multi-threaded, embora tenha adicionado complexidade, permitiu cumprir os requisitos da Fase 6 mantendo a corretude do sistema mesmo sob carga concorrente elevada.